

Foundations of DBSCAN

[DBSCAN]

1.0 Content Block

Disadvantages DBSCAN is not entirely deterministic: border points that are reachable from more than one cluster can be part of either cluster, depending on the order the data are processed. For most data sets and domains, this situation does not arise often and has little impact on the clustering result: both on core points and noise points, DBSCAN is deterministic. DBSCAN* is a variation that treats border points as noise, and this way achieves a fully deterministic result as well as a more consistent statistical interpretation of density-connected components. The quality of DBSCAN depends on the distance measure used in the function $\text{regionQuery}(P, \epsilon)$. The most common distance metric used is Euclidean distance. Especially for high-dimensional data, this metric can be rendered almost useless due to the so-called "Curse of dimensionality", making it difficult to find an appropriate value for ϵ . This effect, however, is also present in any other algorithm based on Euclidean distance. DBSCAN cannot cluster data sets well with large differences in densities, since the $\text{minPts}-\epsilon$ combination cannot then be chosen appropriately for all clusters. If the data and scale are not well understood, choosing a meaningful distance threshold ϵ can be difficult. See the section below on extensions for algorithmic modifications to handle these issues.

2.0 Content Block

Original query-based algorithm DBSCAN requires two parameters: ϵ (eps) and the minimum number of points required to form a dense region (minPts). It starts with an arbitrary starting point that has not been visited. This point's ϵ -neighborhood is retrieved, and if it contains sufficiently many points, a cluster is started. Otherwise, the point is labeled as noise. Note that this point might later be found in a sufficiently sized ϵ -environment of a different point and hence be made part of a cluster. If a point is found to be a dense part of a cluster, its ϵ -neighborhood is also part of that cluster. Hence, all points that are found within the ϵ -neighborhood are added, as is their own ϵ -neighborhood when they are also dense. This process continues until the density-connected cluster is completely found. Then, a new unvisited point is retrieved and processed, leading to the discovery of a further cluster or noise. DBSCAN can be used with any distance function (as well as similarity functions or other predicates). The distance function (dist) can therefore be seen as an additional parameter. The algorithm can be expressed in pseudocode as follows:

3.0 Content Block

Relationship to spectral clustering A spectral implementation of DBSCAN is related to spectral clustering in the trivial case of determining connected graph components — the optimal clusters with no edges cut. However, it can be computationally intensive, up to $O(n^3)$. Additionally, one has to choose the number of eigenvectors to compute. For performance reasons, the original DBSCAN algorithm remains preferable to its spectral implementation.

4.0 Content Block

Extensions Generalized DBSCAN (GDBSCAN) is a generalization by the same authors to arbitrary "neighborhood" and "dense" predicates. The ϵ and minPts parameters are removed from the original algorithm and moved to the predicates. For example, on polygon data, the "neighborhood" could be any intersecting polygon, whereas the density predicate uses the polygon areas instead of just the object count. Various extensions to the DBSCAN algorithm have been proposed, including methods for parallelization, parameter estimation, and support for uncertain data. The basic idea has been extended to hierarchical clustering by the OPTICS algorithm. DBSCAN is also used as part of subspace clustering

algorithms like PreDeCon and SUBCLU. HDBSCAN* is a hierarchical version of DBSCAN which is also faster than OPTICS, from which a flat partition consisting of the most prominent clusters can be extracted from the hierarchy.

5.0 Content Block

Apache Commons Math contains a Java implementation of the algorithm running in quadratic time. ELKI offers an implementation of DBSCAN as well as GDBSCAN and other variants. This implementation can use various index structures for sub-quadratic runtime and supports arbitrary distance functions and arbitrary data types, but it may be outperformed by low-level optimized (and specialized) implementations on small data sets. mlpack includes an implementation of DBSCAN accelerated with dual-tree range search techniques. PostGIS includes ST_ClusterDBSCAN – a 2D implementation of DBSCAN that uses R-tree index. Any geometry type is supported, e.g. Point, LineString, Polygon, etc. R contains implementations of DBSCAN in the packages dbscan and fpc. Both packages support arbitrary distance functions via distance matrices. The package fpc does not have index support (and thus has quadratic runtime and memory complexity) and is rather slow due to the R interpreter. The package dbscan provides a fast C++ implementation using k-d trees (for Euclidean distance only) and also includes implementations of DBSCAN*, HDBSCAN*, OPTICS, OPTICSXi, and other related methods. scikit-learn includes a Python implementation of DBSCAN for arbitrary Minkowski metrics, which can be accelerated using k-d trees and ball trees but which uses worst-case quadratic memory. A contribution to scikit-learn provides an implementation of the HDBSCAN* algorithm. pyclustering library includes a Python and C++ implementation of DBSCAN for Euclidean distance only as well as OPTICS algorithm. SPMF includes an implementation of the DBSCAN algorithm with k-d tree support for Euclidean distance only. Weka contains (as an optional package in latest versions) a basic implementation of DBSCAN that runs in quadratic time and linear memory. linfa includes an implementation of the DBSCAN for the rust programming language. Julia includes an implementation of DBSCAN in the Julia Statistics's Clustering.jl package.

6.0 Content Block

MinPts: As a rule of thumb, a minimum minPts can be derived from the number of dimensions D in the data set, as $\text{minPts} \geq D + 1$. The low value of $\text{minPts} = 1$ does not make sense, as then every point is a core point by definition. With $\text{minPts} \leq 2$, the result will be the same as of hierarchical clustering with the single link metric, with the dendrogram cut at height ϵ . Therefore, minPts must be chosen at least 3. However, larger values are usually better for data sets with noise and will yield more significant clusters. As a rule of thumb, $\text{minPts} = 2 \cdot \text{dim}$ can be used, but it may be necessary to choose larger values for very large data, for noisy data or for data that contains many duplicates. **ϵ :** The value for ϵ can then be chosen by using a k-distance graph, plotting the distance to the $k = \text{minPts} - 1$ nearest neighbor ordered from the largest to the smallest value. Good values of ϵ are where this plot shows an "elbow": if ϵ is chosen much too small, a large part of the data will not be clustered; whereas for a too high value of ϵ , clusters will merge and the majority of objects will be in the same cluster. In general, small values of ϵ are preferable, and as a rule of thumb only a small fraction of points should be within this distance of each other. Alternatively, an OPTICS plot can be used to choose ϵ , but then the OPTICS algorithm itself can be used to cluster the data. **Distance function:** The choice of distance function is tightly coupled to the choice of ϵ , and has a major impact on the results. In general, it will be necessary to first identify a reasonable measure of similarity for the data set, before the parameter ϵ can be chosen. There is no estimation for this parameter, but the distance functions needs to be chosen appropriately for the data set. For example, on geographic data, the great-circle distance is often a good choice. OPTICS can be seen as a generalization of DBSCAN that replaces the ϵ parameter with a maximum value that mostly affects performance. MinPts then essentially becomes the minimum cluster size to find. While the algorithm is much easier to parameterize than DBSCAN, the results are a bit more difficult to use, as it will usually produce a hierarchical clustering instead of the simple data partitioning that DBSCAN produces. Recently, one of the original authors of DBSCAN has revisited DBSCAN and OPTICS, and published a refined version of

hierarchical DBSCAN (HDBSCAN*), which no longer has the notion of border points. Instead, only the core points form the cluster.